

Consolidated Evaluation Report

Total Students	10
Evaluated Students	10
Average Score	14.8 / 20
Highest Score	17.4
Lowest Score	10.0
Plagiarism Cases (≥ 0.80)	0

Student Coverage Overview

Roll Number	Score (/20)	Commits	Public	README
711725UAD136	16.0	8	Yes	Yes
711725UAD202	16.0	12	Yes	Yes
711725UAM123	17.4	7	Yes	Yes
711725UAM251	15.8	4	Yes	Yes
711725UCS149	17.2	1	Yes	Yes
711725UCS219	17.0	14	Yes	No
711725UCS247	10.0	21	Yes	Yes
711725UEC233	11.0	2	Yes	No
711725UIT142	14.0	5	Yes	Yes
711725UIT222	13.6	6	Yes	Yes

Student Evaluation Report

Roll Number: 711725UAD136

Repository: <https://github.com/gharshavarhinia-ship-it/miniProjectSourceCode>

Metric	Value
Commit Count	8
Public Repository	Yes
README Present	Yes

Final Score: 16.0 / 20

Overall Remarks: The student has demonstrated exceptional skill in modifying and extending the application. The most notable contribution is the robust and well-implemented data migration logic (`migrateFileIfNeeded``), which handles schema evolution gracefully. Additionally, the introduction of comprehensive summary statistics and a name-based search greatly enhances the application's functionality. The code is well-structured and shows a deep understanding of C file I/O and data management.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Demonstrates significant effort through the implementation of complex file migration logic, new data analysis features (summary statistics), and an additional search capability (by name).

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Strong understanding of file system operations, data persistence, schema evolution, and basic data analysis principles (statistics, searching).
Added Functionality Ideas	5	Excellent ideas for enhancing the application's utility and robustness, including robust data migration, statistical summaries of accounts, and name-based searching.
Code Comprehension	5	High level of comprehension is evident in the robust and multi-faceted <code>migrateFileIfNeeded`</code> function, which correctly handles various file states and formats (empty, new format, old format). The logic for data transformation and file system operations is well-managed.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	The introduction of the <code>slot`</code> field and, more importantly, the <code>migrateFileIfNeeded`</code> function, greatly improves the code's robustness, maintainability, and ability to handle schema changes gracefully. Ensuring the <code>slot`</code> is correctly set during <code>editAccount`</code> is a good detail.
Functional Decomposition	10	The code is very well-organized into distinct, purpose-specific functions (<code>openCreditFile`</code> , <code>migrateFileIfNeeded`</code> , <code>summaryStatistics`</code> , <code>searchByName`</code> , etc.), enhancing readability and maintainability.

Criteria	Score	Remarks
Memory Optimization	0	No explicit memory optimizations were observed, though the current approach is standard and efficient enough for the given problem scope (MAX_RECORDS 100).
Speed Optimization	0	No explicit speed optimizations were observed. The file I/O operations are inherently the performance bottleneck in this type of application.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully added several basic features including a new `slot` identifier for each record, `summaryStatistics` to provide an overview, `searchByName` for flexible searching, and a 'Low Balance!' warning during account display.
Requirement Translation	10	Demonstrated strong ability to translate implicit requirements for data integrity, improved reporting, and flexible search into well-implemented code features. The data migration logic is an excellent example of translating a complex architectural requirement into a robust solution.
Added Functionality Simple	10	The `summaryStatistics` and `searchByName` functions are well-executed simple additions that significantly enhance the application's utility by providing valuable insights and improved data access.
Added Functionality Advanced	20	The `migrateFileIfNeeded` function is an outstanding advanced addition. It showcases a sophisticated understanding of file format evolution, robust data migration strategies, temporary file usage, and handling file system operations (remove, rename) to ensure data integrity across software versions.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAD202

Repository: <https://github.com/Mrudul-P-Manesh/miniProjectSourceCode>

Metric	Value
Commit Count	12
Public Repository	Yes
README Present	Yes

Final Score: 16.0 / 20

Overall Remarks: The student has made excellent modifications to the base code. They introduced several valuable features, significantly improved the program's robustness through comprehensive input validation, and demonstrated a strong understanding of file handling and functional decomposition. The implementation of the `wipeDatabase`` with safety checks and dual-file clearing is particularly noteworthy. The student's work indicates a strong foundation in C programming and an ability to enhance existing systems effectively.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Student demonstrates significant effort by adding multiple new functionalities, improving input robustness, and ensuring proper file initialization.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Good understanding of basic banking system requirements, implementing features like searching, calculating totals, and database management.
Added Functionality Ideas	5	The ideas for searching by name, calculating bank totals, and a robust database wipe are highly relevant and practical for a bank management system. Adding a phone number field is also a practical enhancement.
Code Comprehension	5	Excellent comprehension of C programming, including binary file I/O (<code>fread</code> , <code>fwrite</code> , <code>fseek</code> , <code>rewind</code>), struct usage, string manipulation (<code>strcmp</code>), and comprehensive input validation techniques using <code>scanf</code> and buffer clearing.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Substantial improvement in code robustness through consistent and effective input validation across all user interactions (menu choice, account numbers, transactions, names, confirmation). Initializing the binary file with 100 blank records on first creation is also a crucial improvement for stability and avoiding garbage data.
Functional Decomposition	10	New features (<code>searchByName`</code> , <code>calculateTotals`</code> , <code>wipeDatabase`</code>) are very well encapsulated into distinct functions, showcasing strong modular design principles.

Criteria	Score	Remarks
Memory Optimization	0	No explicit memory optimization observed.
Speed Optimization	0	No explicit speed optimization observed.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Added a 'phone' field to the `clientData` structure, enhancing the information stored per client.
Requirement Translation	10	All added functionalities (search, totals, wipe, phone number) are fully and correctly implemented and integrated into the menu structure as intended.
Added Functionality Simple	10	Implemented 'search by name' and 'calculate bank totals' features, which are valuable, moderately complex additions for data retrieval and reporting.
Added Functionality Advanced	20	The `wipeDatabase` function, with its strong warning, confirmation prompt, and the clearing of both the binary data file (`credit.dat`) and the text report file (`accounts.txt`), demonstrates advanced consideration for data integrity, user safety, and overall system reset capabilities. The consistent robust input validation across the application also contributes significantly here.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAD225

Repository: https://github.com/Priyadharshini-15coder/AIB_25UAD225_PRIYADHARSHINI

Metric	Value
Commit Count	3
Public Repository	Yes
README Present	Yes

Final Score: 8.0 / 20

Overall Remarks: The student has delivered a highly competent and well-structured implementation of a direct-access file-based bank account management system. The code demonstrates strong understanding of C programming fundamentals, including file I/O, data structures, and functional decomposition. Key code improvements include robust error handling, buffer overflow prevention using ``scanf`` width specifiers, and efficient file pointer manipulation. While the solution effectively meets all core requirements, it does not introduce novel features or advanced optimizations beyond what is typically expected for this type of problem.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student has implemented a complete and functional direct-access file management program from what appears to be a minimal base. This demonstrates significant effort in developing the entire application logic.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrates a strong understanding of C file I/O operations (<code>fopen</code> , <code>fclose</code> , <code>fread</code> , <code>fwrite</code> , <code>fseek</code> , <code>rewind</code>), structure definition, and the logical flow required for managing bank account records.
Added Functionality Ideas	3	The student has successfully implemented the standard set of functionalities expected for a bank account management system (create, update, delete, generate report). While these are core requirements rather than 'new' ideas, their correct implementation reflects a clear understanding of the necessary functionalities for the domain.
Code Comprehension	5	The code is well-structured, logical, and correctly implements all intended operations. This indicates excellent comprehension of the C language constructs and the problem domain.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Several good practices contribute to code improvement: 1. Use of <code>`fseek(fPtr, -sizeof(struct clientData), SEEK_CUR);`</code> in <code>`updateRecord`</code> for efficient and precise file pointer repositioning, rather than re-seeking from the beginning. 2. Using <code>`argv[0]`</code> in the <code>`main`</code> function's <code>`fopen`</code> error message for more robust and informative error reporting. 3. Employing field width specifiers (<code>`%14s%9s`</code>) in <code>`scanf`</code> for <code>`lastName`</code> and <code>`firstName`</code> in <code>`newRecord`</code> to prevent potential buffer overflows, enhancing code safety and robustness.

Criteria	Score	Remarks
Functional Decomposition	5	The program is exceptionally well-organized into distinct, modular functions (e.g., <code>`enterChoice`</code> , <code>`textFile`</code> , <code>`updateRecord`</code> , <code>`newRecord`</code> , <code>`deleteRecord`</code>). This enhances readability, maintainability, and promotes code reusability.
Memory Optimization	0	No explicit advanced memory optimization techniques were identified beyond standard C data handling (e.g., fixed-size structs for records, which is inherent to the problem's direct-access file nature).
Speed Optimization	2	The core approach of using direct-access files (<code>`fseek`</code> , <code>`fread`</code> , <code>`fwrite`</code>) is inherently efficient for accessing specific records. The use of <code>`SEEK_CUR`</code> in <code>`updateRecord`</code> provides a minor, specific optimization by reducing the seek distance for writing back to the same record.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	If the base template was minimal, the implementation of the full suite of basic bank account features (adding new accounts, updating balances, deleting accounts, and generating a text report) constitutes the core 'new features' provided by the student's work.
Requirement Translation	5	The code accurately and completely translates the implied requirements of a robust bank account management system into a fully functional and well-structured program.
Added Functionality Simple	0	No additional simple functionalities beyond the core requirements were added (e.g., searching by name, sorting records, password protection, transaction logging).
Added Functionality Advanced	0	No advanced functionalities were added (e.g., network capabilities, graphical user interface, complex reporting with filters).

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAM123

Repository: <https://github.com/dharaneesh-sys/miniProjectSourceCode>

Metric	Value
Commit Count	7
Public Repository	Yes
README Present	Yes

Final Score: 17.4 / 20

Overall Remarks: The student has made substantial and high-quality additions to the record management system. Key new features include an ATM-style debit transaction with robust balance checks, a function to display individual account details, and an advanced reporting mechanism that sorts all active accounts by name and outputs them to a text file. The code demonstrates excellent understanding of C file I/O, data structures, and functional decomposition. Input validation and error handling are well-implemented, making the modifications robust and reliable for the intended scale.

GENERAL

Criteria	Score	Remarks
Self Effort	4	Student demonstrated significant effort by implementing multiple substantial new features and utilities beyond basic CRUD operations.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Excellent understanding of C file I/O operations (fseek, fread, fwrite), record management principles, and transactional logic with appropriate error checks.
Added Functionality Ideas	5	The added functionalities (account search, ATM-style debit transactions, and sorted reports) are highly relevant, practical, and well-conceived for a record management system.
Code Comprehension	5	New features are seamlessly integrated into the existing codebase, demonstrating a strong understanding of the program's structure and utility functions.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Introduced robust input validation with <code>`get_positive_double`</code> and enhanced error handling (<code>`fread`</code> , <code>`fseek`</code> checks) in new functions. Maintained good coding practices throughout.
Functional Decomposition	10	Outstanding functional decomposition. Each new feature (display, debit, sorting, input validation) is implemented as a clear, single-purpose function, significantly enhancing modularity and readability.
Memory Optimization	8	For <code>`MAX_RECORDS`</code> = 100, loading all records into an array for sorting is an acceptable and efficient approach. No significant memory inefficiencies observed for this scale.

Criteria	Score	Remarks
Speed Optimization	8	Direct record access ($O(1)$) for search and debit operations is efficient. The sorting function uses bubble sort ($O(N^2)$), which is acceptable for `MAX_RECORDS` = 100 and not a performance bottleneck.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully added essential basic features like searching and displaying a single account, and a utility for positive number input validation.
Requirement Translation	5	Excellent translation of implied requirements (e.g., ATM transaction, sorted reporting) into concrete and functional code, directly addressing practical use cases.
Added Functionality Simple	9	Implemented a robust debit transaction feature with crucial balance checks, demonstrating a moderately complex and well-handled financial operation.
Added Functionality Advanced	18	Developed an advanced feature (`write_sorted_text_file`) that reads all active records, sorts them by name using a custom comparison, and writes a formatted report to a new text file.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAM209

Repository: <https://github.com/navanithakrishnanb25am-hue/miniProjectSourceCode>

Metric	Value
Commit Count	14
Public Repository	Yes
README Present	Yes

Final Score: 9.6 / 20

Overall Remarks: The student demonstrated significant effort and strong conceptual understanding by adding a comprehensive set of features, including a secure login system with masked input and attempt limits, user management (add/change password), inter-account money transfer with validation, and direct record search. The functional decomposition is excellent, and efficient file I/O using `fseek` was correctly applied for performance. However, a critical recurring issue of missing curly braces in conditional statements (`if` blocks) led to significant logical and structural bugs throughout the code. This fundamental C syntax error severely impacted code quality and comprehension, despite the ambitious and well-conceived feature set. Addressing these structural flaws would significantly improve the robustness and correctness of the implementation.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Implemented a robust login system, user management, and core banking functionalities like money transfer and account search, demonstrating substantial effort and ambition.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Exhibits strong domain knowledge in implementing secure login, user management (admin access, password change), and transactional banking logic (balance checks for transfers) with appropriate safeguards.
Added Functionality Ideas	5	The chosen functionalities (login, transfer, user management, search, file initialization) are highly relevant, well-conceived, and significantly enhance the application's utility and realism.
Code Comprehension	2	Understood complex file operations (<code>fseek</code> , <code>remove</code> , <code>rename</code>) and algorithm design for features. However, exhibited a fundamental misunderstanding of C's conditional block syntax (recurrent missing curly braces), leading to multiple logical errors in execution flow.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	1	While some user experience improvements (like password masking with <code>getch()</code>) were made, the pervasive issue of missing braces in conditional statements resulted in significant logical flaws and poor structural integrity in multiple parts of the code.

Criteria	Score	Remarks
Functional Decomposition	5	Excellent functional decomposition; each new feature (login, search, transfer, initializeFile, addUser, changePassword) is encapsulated in a dedicated and clearly defined function, promoting modularity.
Memory Optimization	0	No specific memory optimization techniques were applied or explicitly necessary for the scope of the modifications.
Speed Optimization	5	Utilized `fseek` for direct record access in search and transfer operations, which is a significant speed optimization compared to sequential file reads, especially for larger data files.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully implemented essential basic features like account search functionality and a utility for file initialization.
Requirement Translation	5	Demonstrated strong ability to translate complex conceptual requirements, such as secure user authentication, inter-account funds transfer with validation, and user profile management, into functional code.
Added Functionality Simple	5	Added straightforward but valuable functionalities such as administrative user addition and file initialization, contributing to system maintainability and setup.
Added Functionality Advanced	5	Implemented sophisticated features like a secure login system with password masking and attempt limits, a robust inter-account money transfer mechanism with validation, a password change mechanism using temporary files, and role-based access control (admin check).

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UAM251

Repository: https://github.com/SARATHI-78/miniProjectSourceCode_251

Metric	Value
Commit Count	4
Public Repository	Yes
README Present	Yes

Final Score: 15.8 / 20

Overall Remarks: The student has significantly enhanced the banking application by introducing a robust set of new features including a basic login system, money transfer, transaction logging, history, interest application, mini statements, and account viewing. The code is well-structured with excellent functional decomposition. The 'logTransaction' function is a particularly good addition, improving traceability and enabling features like transaction history and mini statements. The 'transferMoney' function is also well-implemented with appropriate checks for invalid accounts and insufficient balance, demonstrating strong problem-solving skills and a clear understanding of the domain.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Demonstrated significant effort by adding a comprehensive suite of new features and integrating them effectively into the existing banking system.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Good understanding of banking domain concepts, evident in the implementation of features like money transfer, interest application, and transaction logging.
Added Functionality Ideas	5	Excellent ideas for added functionality that greatly enhance the utility and realism of the banking application, moving beyond basic CRUD operations.
Code Comprehension	5	Strong understanding of how to modify and extend the existing codebase using file I/O operations and C language constructs for new features.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Improved the base code by integrating a transaction logging mechanism into existing update, add, and delete operations, enhancing traceability.
Functional Decomposition	10	Outstanding functional decomposition. Each new feature is implemented as a separate, clearly defined function, contributing to modularity and readability.
Memory Optimization	0	No explicit memory optimization techniques were observed in the added code.
Speed Optimization	0	No explicit speed optimization techniques were observed in the added code.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully implemented multiple essential basic features, including a login system, viewing account details, full transaction history, and applying interest to accounts.
Requirement Translation	9	Effectively translated implicit requirements for a more complete banking system into practical and functional code features.
Added Functionality Simple	10	Developed several simple yet impactful functionalities like logging transactions with timestamps, displaying transaction history, and generating mini-statements for specific accounts.
Added Functionality Advanced	20	The 'transferMoney' function is a robust and complex addition, handling multiple account updates, balance checks, and integrated logging, demonstrating advanced functionality.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UCS148

Repository: <https://github.com/Karkkiniyan18/miniProjectSourceCode>

Metric	Value
Commit Count	6
Public Repository	Yes
README Present	Yes

Final Score: 9.0 / 20

Overall Remarks: The student successfully implemented a new 'Display all accounts' feature. This involved creating a new `displayAll` function, integrating it into the main menu, and correctly using file I/O to read and format account data. The functional decomposition is good, and the student demonstrated solid comprehension of file handling and data structures. The addition significantly improves the application's basic functionality.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student identified a common missing feature (displaying all records) and implemented it, showing good self-effort.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrated good understanding of file I/O operations (<code>fread</code> , <code>rewind</code> , <code>fopen</code>), <code>struct</code> usage, and formatted printing to implement the display functionality.
Added Functionality Ideas	5	The idea to add a 'display all accounts' feature is practical and a common requirement for such an application.
Code Comprehension	5	Correctly understood how to integrate the new functionality into the existing menu system and leverage the file pointer for reading records.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	0	No explicit improvements were made to existing code segments; the changes involve adding new functionality.
Functional Decomposition	5	The new 'displayAll' logic is well-encapsulated in its own function, demonstrating good functional decomposition.
Memory Optimization	0	Not applicable to the added functionality.
Speed Optimization	0	Not applicable to the added functionality.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	A clear, new basic feature allowing the user to display all existing accounts from the data file.
Requirement Translation	5	Effectively translated the idea of 'viewing all accounts' into a working code implementation with proper formatting.
Added Functionality Simple	10	The 'display all' functionality is a simple yet essential addition that significantly enhances the application's usability.
Added Functionality Advanced	0	The implemented feature, while useful, is not considered advanced.

PLAGIARISM CHECK

Other Roll	Similarity
711725UEC233	0.85

Student Evaluation Report

Roll Number: 711725UCS149

Repository: <https://github.com/kavyaparthiban-ship-it/cminiproject>

Metric	Value
Commit Count	1
Public Repository	Yes
README Present	Yes

Final Score: 17.2 / 20

Overall Remarks: The student has significantly enhanced the banking application by introducing three highly relevant and well-implemented new features: fund transfer between accounts, a global account audit, and searching by full name. The `transferFunds` function, in particular, demonstrates a strong understanding of transactional logic, robust error handling, and correct file I/O operations for updating multiple records. The use of `scanf` with width specifiers for input is a good practice for preventing buffer overflows. The code is well-structured and demonstrates excellent comprehension of C programming concepts and file handling. The modifications show a high level of self-effort and a deep understanding of the problem domain.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Implemented three significant new features, demonstrating high self-effort in understanding requirements and applying C file I/O concepts.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrated a strong grasp of banking domain concepts, including fund transfers, account audits, and account search, incorporating relevant validations and calculations.
Added Functionality Ideas	5	The selected new functionalities (fund transfer, account audit, search by name) are practical, well-conceived, and significantly enhance the application's utility.
Code Comprehension	5	Clearly understood how to interact with file-based data structures, manage file pointers (fseek, fread, fwrite), and seamlessly integrate new features into the main program flow.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Implemented robust error handling for transfer operations (e.g., insufficient funds, non-existent accounts) and used safer input methods (<code>scanf</code> with width specifiers) to prevent buffer overflows.
Functional Decomposition	10	Excellent modular design with each new feature neatly decomposed into separate, well-defined functions (<code>transferFunds</code> , <code>accountAudit</code> , <code>searchByFullName</code>).

Criteria	Score	Remarks
Memory Optimization	3	No specific memory optimizations were implemented, but the current approach of handling fixed-size structs directly in file I/O is inherently efficient for this task.
Speed Optimization	3	No explicit speed optimizations were implemented; operations like audit and search involve linear file scans, which is standard for unindexed flat files.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully added a basic yet essential feature for searching accounts by full name, including basic output formatting.
Requirement Translation	10	Demonstrated strong ability to translate functional requirements (like transferring funds or auditing accounts) into well-structured and working C code.
Added Functionality Simple	10	Implemented a useful 'account audit' feature that calculates total bank deposits and identifies the client with the highest balance.
Added Functionality Advanced	20	Developed an advanced 'transfer funds' feature with complex logic, including robust source/destination account validation, balance checks, and atomic updates to two separate records in the file.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UCS219

Repository: https://github.com/RohithMummoorthy/711725UCS219_CSE-B_-C-Mini-project

Metric	Value
Commit Count	14
Public Repository	Yes
README Present	No

Final Score: 17.0 / 20

Overall Remarks: This project demonstrates an exceptionally high level of skill, effort, and understanding. The student has transformed a basic `credit.dat` program into a fully-featured, robust, and well-designed Banking Management System. The breadth of functionality, including a multi-role login system, varied account types, fixed deposits, comprehensive transaction logging, ATM simulation, and system utilities like backup/restore and search/sort, is truly impressive. The code is modular, well-commented, and includes excellent input validation and error handling, making it a professional-grade student project. This submission significantly exceeds expectations for student-added modifications.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Demonstrates exceptional effort in building a comprehensive banking system from a simpler base. The sheer volume of features, detailed structures, and well-organized code indicate a very high level of dedication and hard work.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Excellent understanding of banking domain knowledge, evidenced by the implementation of various account types, fixed deposits, interest calculation, service charges, secure login, and robust transaction logging.
Added Functionality Ideas	5	Showcases highly creative and practical ideas for functionality, transforming a basic program into a full-fledged banking management system with ATM simulation, backup, and extensive reporting.
Code Comprehension	5	The code is exceptionally well-structured, modular, and extensively commented, making it easy to understand and follow the logic of complex features. Good use of specific data types and constants.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Major improvements in code quality, including robust input validation (safeReadInt, safeReadDouble, validateDOB, validatePhone), modular design, clear structure definitions, and comprehensive error handling for file operations and user input.
Functional Decomposition	10	Outstanding functional decomposition, breaking down complex banking operations into many well-defined, single-purpose functions, which greatly enhances maintainability and readability.

Criteria	Score	Remarks
Memory Optimization	5	Utilizes binary files (`.dat`) for efficient storage and retrieval of structured data, which is a good choice for this type of application in C, contributing positively to memory usage compared to plain text files for direct record access.
Speed Optimization	5	The use of binary files helps in faster data access and processing compared to parsing large text files, providing a good baseline for performance in a C application of this nature.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully implemented basic but crucial new features like a foundational login system and core deposit/withdrawal functionalities.
Requirement Translation	5	Excellent ability to translate a broad set of banking requirements, from user authentication to complex financial products like FDs and transaction logging, into a functional and robust system.
Added Functionality Simple	10	Implemented a wide array of simple yet highly effective functionalities, including automatic age calculation from DOB, comprehensive date utilities, robust input validation, mini statements, and detailed account summaries.
Added Functionality Advanced	20	Exceptional implementation of advanced functionalities such as a multi-role login system with account locking, diverse account types (Savings, Current, Minor), a complete Fixed Deposit management module with maturity calculations, comprehensive transaction logging, fund transfer, a full ATM simulation, and critical system features like backup/restore, search, and sort.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UCS247

Repository: https://github.com/Tarunika-Muruganathan/711725UCS247_CSE-B_C-Mini-project

Metric	Value
Commit Count	21
Public Repository	Yes
README Present	Yes

Final Score: 10.0 / 20

Overall Remarks: The student has made outstanding modifications to the banking system. The additions cover crucial areas such as security (admin login, PIN-based account access), data enrichment (expanded client profile), core banking operations (deposit, withdraw, transfer), and robust auditing (transaction logging with timestamps and viewing). The code demonstrates strong domain knowledge, excellent functional decomposition, and a clear commitment to delivering a more secure and feature-rich application. The consistent implementation of PIN checks across all sensitive functions is particularly commendable for enhancing security. While memory and speed optimizations were not evident, the focus on functionality, security, and data integrity is highly impressive.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student has demonstrated significant self-effort by implementing a wide range of new features and improvements, indicating a thorough understanding and commitment to enhancing the system.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Excellent understanding of banking domain knowledge, evident in the implementation of admin login, PIN-based security, deposit, withdrawal, transfer functionalities, transaction logging, and detailed client attributes (gender, DOB, nationality, account type).
Added Functionality Ideas	5	The ideas for added functionality are highly relevant and well-conceived, covering critical aspects like security (admin login, PIN), core transactions (deposit, withdraw, transfer), data integrity (DOB validation, balance checks), and auditing (transaction log).
Code Comprehension	5	The student demonstrates strong code comprehension by effectively integrating new features, correctly handling file I/O operations (`fseek`, `fread`, `fwrite`), string manipulations, and consistently applying security checks (PIN) across multiple relevant functions. The `isValidDOB` logic is well-structured.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Significant code improvements include adding robust PIN-based authentication to all sensitive operations (update, deposit, withdraw, transfer, search), implementing `isValidDOB` for data validation, and enhancing `displayAll` to generate a formatted file report. Overdraft/insufficient balance checks also improve data integrity.

Criteria	Score	Remarks
Functional Decomposition	5	The student has excellent functional decomposition, creating distinct functions for `login`, `isValidDOB`, `deposit`, `withdraw`, `transfer`, `logTransaction`, `viewTransactions`, and `viewTransactionsByAccount`, which improves modularity and readability.
Memory Optimization	0	No explicit memory optimization techniques (e.g., dynamic memory allocation, optimized data structures for memory footprint) were observed in the provided code snippet.
Speed Optimization	0	No explicit speed optimization techniques were observed. File I/O operations are standard, and algorithms for searching/updating are direct access based on account number, which is efficient for this context.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Basic new features include an admin login, expanded client data model (gender, DOB, nationality, account type), `isValidDOB` validation, and core deposit/withdraw functionalities. These form a strong foundation for the system.
Requirement Translation	5	The student effectively translated common banking system requirements into functional code, addressing security, customer data management, transaction processing, and basic reporting needs comprehensively.
Added Functionality Simple	5	Simple but impactful added functionalities include inter-account `transfer`, consistent PIN-based authentication for all transactions and searches, and generating a formatted `accounts.txt` report from `displayAll`.
Added Functionality Advanced	5	Advanced functionalities include a full-fledged transaction logging system (`logTransaction`) with timestamps and dedicated functions to `viewTransactions` and `viewTransactionsByAccount`. This demonstrates sophisticated system design beyond basic CRUD operations.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UCY118

Repository: <https://github.com/iyyappan100808-glitch/miniProjectSourceCode>

Metric	Value
Commit Count	4
Public Repository	Yes
README Present	Yes

Final Score: 8.2 / 20

Overall Remarks: The student demonstrated effort in implementing a menu-driven banking transaction system, including deposit, withdrawal, and a specific business rule for minimum balance. The conceptual understanding of these banking operations is fair. However, the code suffers from critical syntax and logical errors, primarily due to the omission of curly braces for ``if`` and ``else if`` statements that contain multiple lines of code. This fundamental misunderstanding of C's block scope leads to incorrect control flow, making the deposit and withdrawal logic behave unexpectedly, and causing 'Invalid option' messages to appear even for valid selections. The attempt to add functionality is commendable, but the execution highlights a significant gap in foundational C programming knowledge.

GENERAL

Criteria	Score	Remarks
Self Effort	3	Student made a clear attempt to implement a transaction menu with deposit, withdrawal, and a minimum balance check. However, the implementation has critical errors.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrates a good understanding of basic banking operations (deposit, withdraw) and a specific business rule (minimum balance of 500).
Added Functionality Ideas	5	The ideas for adding deposit, withdrawal, minimum balance check, user input, and balance display are relevant and constitute core functionalities for a basic banking system.
Code Comprehension	3	Shows basic understanding of <code>`printf`</code> , <code>`scanf`</code> , and arithmetic operations. However, there's a severe lack of comprehension regarding C's control flow block structure (missing curly braces for <code>`if`</code> and <code>`else if`</code> statements), leading to incorrect logic and execution path.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	While new features are added, the code introduces significant bugs due to fundamental syntax errors (missing curly braces), making it functionally incorrect. This doesn't qualify as an improvement in terms of code quality or correctness.
Functional Decomposition	4	The code implements a sequence of operations within what appears to be a single functional block. There's no evident attempt at further decomposition into smaller, specialized functions.

Criteria	Score	Remarks
Memory Optimization	0	No specific memory optimization attempts were made.
Speed Optimization	0	No specific speed optimization attempts were made.

INNOVATION

Criteria	Score	Remarks
New Features Basic	4	Basic features like deposit, withdrawal, user interaction for amount and option, and a minimum balance check for withdrawals are added.
Requirement Translation	5	The student correctly identified the requirements for deposit, withdrawal, and a minimum balance rule. However, the translation of these requirements into correctly functioning C code is flawed due to critical implementation errors.
Added Functionality Simple	7	Successfully attempts to implement several simple functionalities, including user input and conditional logic for transactions, even though the implementation has significant errors.
Added Functionality Advanced	0	No advanced functionalities beyond basic transactions and a simple rule were implemented.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UEC163

Repository: <https://github.com/mutharasu-kannan/MINIPROJECT-C>

Metric	Value
Commit Count	9
Public Repository	Yes
README Present	Yes

Final Score: Not available

Overall Remarks: Evaluation not available for this student.

Section-wise evaluation details are not available.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UEC215

Repository: <https://github.com/praveenec-info/Mini-project-C>

Metric	Value
Commit Count	3
Public Repository	Yes
README Present	No

Final Score: Not available

Overall Remarks: Evaluation not available for this student.

Section-wise evaluation details are not available.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UEC233

Repository: <https://github.com/sanjaykumars25ec-design/Sanjaysspointer>

Metric	Value
Commit Count	2
Public Repository	Yes
README Present	No

Final Score: 11.0 / 20

Overall Remarks: The student has developed a strong and well-structured banking system. Key modifications include enriching the ``clientData`` with new fields, implementing robust CRUD operations with business logic (like minimum balance enforcement), and adding a valuable data export feature to a text file. The code demonstrates good understanding of file I/O and functional decomposition. While advanced optimizations or complex algorithms were not a focus, the student delivered a highly functional and practical application for the given scope.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student has put in significant effort to implement a complete banking system with multiple features and business rules using C file I/O.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrated good understanding of banking domain concepts such as account numbers, types, balances, transactions (deposit/withdraw), and minimum balance rules.
Added Functionality Ideas	5	Excellent ideas for added functionalities, including more detailed client data (account type, phone), exporting data to a text file, and enforcing minimum balance rules for transactions and account creation.
Code Comprehension	5	The student shows good comprehension of C file I/O operations (<code>fread</code> , <code>fwrite</code> , <code>fprintf</code>), structured data, and menu-driven program flow. Minor logical inconsistencies like the misplaced 'Invalid choice' print statement in <code>main()</code> for a handled case could be improved.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	4	Improved the <code>`clientData`</code> structure by adding relevant fields (<code>`accType`</code> , <code>`phone`</code>) and integrated these fields across all operations (add, update, display, save to text). The addition of business logic (minimum balance) makes the system more robust.
Functional Decomposition	5	Excellent functional decomposition, with clear separation of concerns into distinct functions for adding, updating, deleting, displaying accounts, and saving data to a text file.
Memory Optimization	0	No explicit memory optimization techniques were identified.

Criteria	Score	Remarks
Speed Optimization	0	No explicit speed optimization techniques were identified.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Successfully implemented basic but crucial new features such as adding account type and phone number fields, and a menu option to save all account data to a text file.
Requirement Translation	8	The student effectively translated general requirements for a banking system into specific functions and added practical features like minimum balance enforcement, showing good foresight in application development.
Added Functionality Simple	8	The 'Save all to Text File' functionality is a great simple addition, providing a human-readable export of data. The minimum balance checks during account creation and update also count as valuable simple functionalities.
Added Functionality Advanced	5	While the overall system is comprehensive and well-executed for a basic C file I/O application, it does not include advanced concepts like complex algorithms, multi-threading, or intricate data structures beyond simple structs. The focus is on robust implementation of core features.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UIT142

Repository: <https://github.com/kalaivani-venkatesan/Kalaivani-TPS>

Metric	Value
Commit Count	5
Public Repository	Yes
README Present	Yes

Final Score: 14.0 / 20

Overall Remarks: The student has delivered an outstanding implementation, demonstrating exceptional initiative and skill in developing a banking system. Key strengths include significant security enhancements (PIN hashing, account locking), comprehensive transaction management (daily limits, detailed logging), and well-structured user and administrative interfaces. The functional decomposition and code improvements are highly commendable. While some admin panel functions are presented as menu options without full implementation details in the provided snippet, the overall design, integration of features, and attention to detail are excellent, making this a highly impressive submission.

GENERAL

Criteria	Score	Remarks
Self Effort	5	The student demonstrates significant self-effort by implementing a wide array of new features and improvements, going well beyond basic requirements for a banking system.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Excellent understanding of banking domain concepts, including account creation, secure login (PIN hashing, account locking), transaction types (deposit, withdrawal, transfer), daily limits, and administrative controls.
Added Functionality Ideas	5	A rich set of functional ideas are present, such as PIN hashing, 3-attempt login with account locking, daily withdrawal limits, detailed transaction logging, receipt generation, and a comprehensive admin panel menu with various account management options.
Code Comprehension	5	The student exhibits strong code comprehension by structuring the application into logical modules (account, transaction, admin, utils), managing file I/O for data persistence, and integrating various functionalities coherently.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Numerous code improvements were added: PIN hashing for security, account locking mechanism, daily withdrawal limit enforcement, robust transaction logging, basic input validations (e.g., negative balance/amount), and efficient in-place file updates using `fseek` for account modifications.

Criteria	Score	Remarks
Functional Decomposition	10	Excellent functional decomposition is evident with clearly defined functions for distinct tasks like <code>createAccount</code> , <code>login</code> , <code>deposit</code> , <code>withdraw</code> , <code>logTransaction</code> , <code>adminPanel</code> , <code>hashPin</code> , <code>clearScreen</code> , <code>pauseScreen</code> , and <code>generateReceipt</code> . This promotes modularity and readability.
Memory Optimization	0	No explicit memory optimization techniques were observed beyond standard C practices. The code uses fixed-size structs, which is appropriate for this application type.
Speed Optimization	0	No specific speed optimization techniques were implemented. File I/O operations could potentially be optimized for very large datasets, but for typical banking system scale, the current approach is acceptable.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	Basic new features include PIN hashing, a 3-attempt login system with account locking, and a daily withdrawal limit. These significantly enhance the system's security and functionality.
Requirement Translation	10	The student has excellently translated common banking system requirements into a functional system with intuitive user and admin menus, reflecting a clear understanding of system needs.
Added Functionality Simple	10	Simple but effective functionalities such as clear/pause screen utilities (<code>clearScreen</code> , <code>pauseScreen</code>) for improved user experience and comprehensive menu structures for both user dashboard and admin panel are well-implemented.
Added Functionality Advanced	10	Advanced functionalities include a robust, timestamped transaction logging system, a persistent account locking mechanism, and the conceptual design of a full-fledged admin panel with diverse account management options (even if some calls are placeholders).

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725UIT219

Repository: <https://github.com/Ragavendhar25/miniProjectSourceCode>

Metric	Value
Commit Count	5
Public Repository	Yes
README Present	Yes

Final Score: 8.8 / 20

Overall Remarks: The student successfully added a new 'search record' feature to the existing file management system. This modification includes the declaration of the ``searchRecord`` function, its implementation using direct file access (``fseek``, ``fread``), and integration into the main menu. The implementation demonstrates a solid understanding of file I/O and structured data manipulation. The new feature is well-encapsulated, functional, and enhances the system's utility. While the functionality is basic, it is correctly implemented and adds significant value.

GENERAL

Criteria	Score	Remarks
Self Effort	4	The student successfully implemented a new 'search record' feature, demonstrating a clear understanding of integrating new functionality into an existing menu-driven file management system.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	4	Demonstrated good domain knowledge in file handling operations such as <code>`fseek`</code> , <code>`fread`</code> , and managing file pointers for direct access in a structured record file.
Added Functionality Ideas	5	The idea of adding a 'search account' feature is highly practical and a logical extension for any record management system, enhancing its usability significantly.
Code Comprehension	4	The student correctly comprehended the existing file structure, record size, and direct access pattern to efficiently implement the new search functionality.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	0	No specific improvements to existing base/template code were observed; the modifications were additions of new functionality.
Functional Decomposition	5	The new 'search record' logic is well-encapsulated within its own function (<code>`searchRecord`</code>), adhering to good functional decomposition principles.
Memory Optimization	0	No explicit memory optimizations were introduced as part of the new feature.

Criteria	Score	Remarks
Speed Optimization	1	While the direct access search by account number is inherently efficient ($O(1)$ lookup), this is due to the established fixed-record file structure, not an explicit optimization applied by the student over a slower existing method.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	The 'search account by number' is a fundamental and valuable new feature for a basic record management system.
Requirement Translation	8	The implied requirement for a 'search' feature was accurately translated into a functional code implementation that correctly locates and displays account information.
Added Functionality Simple	8	The implementation of searching by a primary key (account number) is a straightforward yet crucial functionality for interacting with the data file.
Added Functionality Advanced	0	The implemented search functionality, while effective, does not incorporate advanced algorithms or complex data structures.

PLAGIARISM CHECK

Other Roll	Similarity
711725UIT222	0.81

Student Evaluation Report

Roll Number: 711725UIT222

Repository: <https://github.com/ravirajv25it/miniProjectSourceCode>

Metric	Value
Commit Count	6
Public Repository	Yes
README Present	Yes

Final Score: 13.6 / 20

Overall Remarks: The student has made significant and commendable improvements to the base code, primarily focusing on enhancing robustness, efficiency, and adhering to good programming practices. The introduction of ``clearInputBuffer`` addresses a critical aspect of C console programming, and the file I/O optimizations demonstrate a deep understanding of ``fseek`` mechanics. Overall, a very strong submission with practical and well-executed modifications.

GENERAL

Criteria	Score	Remarks
Self Effort	5	Student clearly invested significant effort in understanding and addressing common C programming pitfalls (like input buffer issues) and optimizing file I/O operations.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Demonstrates a strong understanding of C file handling functions (<code>`fseek`</code> , <code>`fread`</code> , <code>`fwrite`</code>), struct usage, and input/output stream management.
Added Functionality Ideas	5	The ideas for improving input robustness (<code>`clearInputBuffer`</code>) and optimizing file access patterns are practical and show good foresight.
Code Comprehension	5	Successfully identified areas for improvement within the existing codebase and integrated modifications effectively and logically.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	5	Excellent improvements, including the <code>`clearInputBuffer`</code> function for robust input, the use of the <code>`BLANK_CLIENT`</code> constant for efficiency, and optimized <code>`fseek`</code> patterns for update/delete/create operations.
Functional Decomposition	8	The addition of the <code>`clearInputBuffer`</code> helper function is a good example of functional decomposition, encapsulating a specific utility.
Memory Optimization	5	Minor memory optimization achieved through the use of the <code>`BLANK_CLIENT`</code> global constant, reducing the need for repeated temporary struct creation on the stack during deletions.
Speed Optimization	10	The optimized <code>`fseek`</code> patterns (using <code>`SEEK_CUR`</code> after <code>`fread`</code> for writing back to the same record) directly improve the speed of file operations by reducing redundant absolute seeks.

INNOVATION

Criteria	Score	Remarks
New Features Basic	5	The `clearInputBuffer` function is a valuable new basic utility that significantly enhances user experience and program robustness in console applications.
Requirement Translation	10	Effectively translated implicit requirements for a robust and efficient console application into concrete, well-implemented code modifications.
Added Functionality Simple	5	The `clearInputBuffer` function provides a simple yet crucial functionality for reliable input processing, preventing common input-related bugs.
Added Functionality Advanced	0	No advanced new functionalities were implemented.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.

Student Evaluation Report

Roll Number: 711725URA111

Repository: <https://github.com/vpchandru-create/sample-pro>

Metric	Value
Commit Count	2
Public Repository	Yes
README Present	Yes

Final Score: 3.2 / 20

Overall Remarks: The student has correctly implemented a fundamental file seek operation using ``fseek`` with a negative offset based on ``sizeof`` a structure, demonstrating a solid grasp of binary file manipulation concepts. While this single line itself isn't a complex feature, it's a vital building block for functionalities such as re-reading the last record or enabling a simple 'undo' mechanism. Without further context, its contribution to larger features or improvements cannot be fully assessed.

GENERAL

Criteria	Score	Remarks
Self Effort	2	The student has correctly implemented a file seek operation using a negative offset and <code>`sizeof`</code> , which requires understanding of binary file I/O, structs, and pointer arithmetic. This demonstrates a level of effort beyond simple boilerplate.

COMPREHENSION

Criteria	Score	Remarks
Domain Knowledge	5	Shows good understanding of binary file manipulation, specifically the <code>`fseek`</code> function, <code>`sizeof`</code> operator, and relative file positioning (<code>`SEEK_CUR`</code>). The usage is appropriate and correct for navigating structured data in a file.
Added Functionality Ideas	0	This line is an implementation detail, not an expression of an idea for functionality.
Code Comprehension	4	The line is clear, concise, and correctly written. Its purpose is readily understandable for anyone familiar with C file I/O.

MODIFICATION

Criteria	Score	Remarks
Code Improvement	0	Without prior code or context, it's impossible to determine if this line serves as an improvement to existing logic. It's evaluated as new logic rather than an improvement.
Functional Decomposition	0	A single line of code does not demonstrate functional decomposition.
Memory Optimization	0	This specific line of code is not an optimization technique but a standard file navigation operation.
Speed Optimization	0	This specific line of code is not an optimization technique but a standard file navigation operation.

INNOVATION

Criteria	Score	Remarks
New Features Basic	3	This line, if newly added by the student, represents a fundamental piece of logic required for basic features such as re-reading the last written record, or implementing a simple 'undo' mechanism for file operations. It's a building block for functionality.
Requirement Translation	0	No requirements were provided, so translation cannot be assessed.
Added Functionality Simple	2	Contributes to enabling a simple, low-level functionality within a file-based application, such as stepping back one record.
Added Functionality Advanced	0	This single line does not represent advanced functionality.

PLAGIARISM CHECK

No high-similarity plagiarism matches found.